

Материалы кафедры ММП факультета ВМК МГУ. Введение в глубокое обучение.

Занятие 11. Эмбединги слов, дистрибутивная гипотеза, word2vec

Занятие провёл: Феокистов Дмитрий
(@trandelik)

Материалы составили Денисов Егор (mail:
denisov.official72@gmail.com), Ким
Роман (tg: @karman_rim), Оганов
Александр (tg: @welmud)

Москва, Весенний семестр 2026

Источники:

- Часть материалов основана на курсе [Математические методы анализа текстов, кафедра ММП](#)
- [Конспект по основам NLP](#), который подготовили: Феокистов Дмитрий, Хисматуллин Владимир
- Часть материалов основана на [NLP Course](#)
- **Distributional Structure:** [Zellig S. Harris, 1954](#), примерно 8400 цитирований
- **A Synopsis of Linguistic Theory:** [J. Firth, 1957](#), примерно 5700 цитирований
- **Count-based methods:** [Peter D. Turney and Patrick Pantel, 2010](#), примерно 4200 цитирований
- **GloVe:** [Jeffrey Pennington et al., 2014](#), примерно 50 тысяч цитирований

- **Word2Vec:** [Tomas Mikolov et al., 2013](#), примерно 52 тысячи цитирований
- **Negative sampling and other acceleration methods:** [Tomas Mikolov et al., 2013](#), примерно 50 тысяч цитирований
- **FastText:** [Piotr Bojanowski et al., 2017](#), примерно 15 тысяч цитирований
- Для большинства изображений указан источник, откуда оно взято, как правило, являющийся статьей

В этом ноутбуке мы постараемся познакомиться с задачей обработки естественного языка (natural language processing/NLP). В частности на данной лекции будут разобраны методы получения векторных представлений слов – от простых статистических до нейросетевых подходов. Кроме того, вспомним о том, что такая токенизация, и какой она бывает.

Оффтоп: не стоит думать, что NLP ограничивается только большими моделями, в начале развития были использованы **колоссальные труды** лингвистов и людей с филологическим образованием. Гигантские команды формулировали различные логические правила на основе знаний о синтаксисе языка. Например, вы можете изучить [национальный корпус русского языка](#), в котором собирались знания многих специалистов о русском языке. Каждый текст имеет: стиль, автора, год, при этом корпус содержит более 2 миллиардов слов (что в **10000 раз больше**, чем в книге "Война и мир").

Примеры основных задач NLP на практике

В области обработки языка существуют множество различных задач, но все их можно поделить на **2 категории**:

- Дискриминативные (классификация, распознавание сущностей, похожесть текстов...)
- Генеративные (генерация текстов, условная генерация: перевод, чат-боты...)

Более подробная схема изображена ниже.

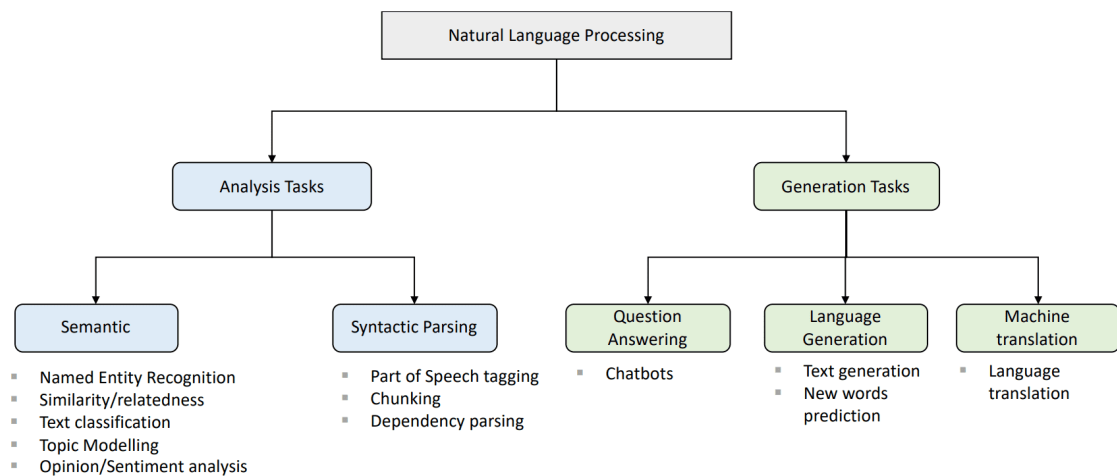


Figure 5: NLP Tasks Categorized in Two Broader Categories: Analysis Tasks (light blue) and Generation Tasks (light green)

Источник

Способы токенизации текста

В любой задаче машинного обучения мы строим некоторую модель, которая принимает на вход данные в числовом формате и возвращает числовой ответ. Обработка естественного языка не является исключением, поэтому нам очень важно решить задачу **перевода текста в числовое представление**.

Очевидно, что кодировать текст некоторым вектором целиком не является оптимальным решением – такой вектор будет содержать слишком мало информации, и, кроме того, различных текстов бесконечно много. Таким образом, прежде чем переводить текст в числовое представление, нам необходимо разбить его на смысловые единицы (токены), которые и будут обрабатываться нашей моделью. Существует несколько подходов к такому разбиению (которое, кстати, называется токенизацией).

Character-level

В качестве токенов будем брать отдельные символы.

Плюсы:

- Очень маленький размер словаря
- Легко кодировать любые слова – нет проблемы с out of vocabulary

Минусы:

- Длинные последовательности. Почему это плохо, если мы фактически получаем больше фичей? В данном случае наши признаки несут в себе очень маленькие

- куски информации об исходном тексте, и модели будет тяжело ее извлечь, так как нужно выучить сложные зависимости между токенами. Плюс это накладывает вычислительные расходы из-за того, что мы увеличиваем размер входных данных
- Теряется языковая структура

Word-level

В качестве токенов будем брать слова целиком.

Плюсы:

- Последовательности токенов после такого разбиения будут получаться достаточно короткими + более репрезентативные признаки
- Сохраняется структура языка

Минусы:

- Большой размер словаря
- Нет токенов для новых слов, не входящих в наш словарь (частичное решение проблемы – завести специальный токен <UNK> для новых слов, но такой токен не будет нести никакой смысловой нагрузки)

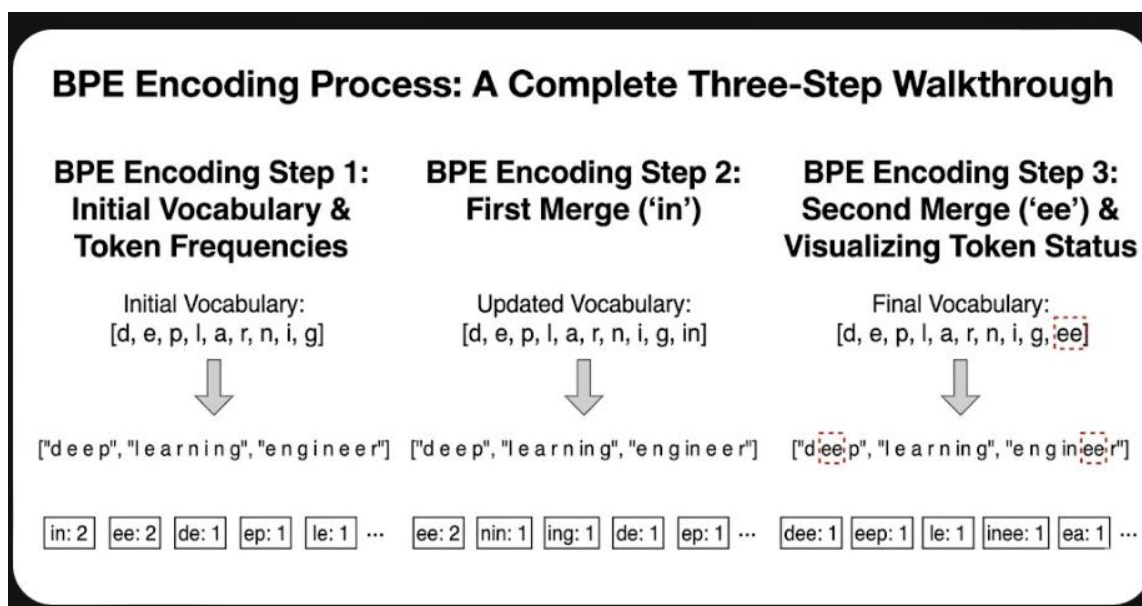
Byte-pair encoding (BPE)

В качестве компромисса между предыдущими методами используется следующий алгоритм:

Основная идея: Будем строить словарь иерархически.

1. В качестве исходного словаря используются символы ← `character-level` ;
2. Подсчитываются частоты встречаемости всех пар символов;
3. Находится самая частая пара;
4. Данная пара добавляется в наш словарь;
5. Если размер словаря стал больше, чем изначально заданный, то заканчиваем. Иначе идем в пункт 2.

По итогу получаем словарь, где мы учитываем популярные сочетания букв/слов/слов, причем используются исходные данные и знания о них. Получается, что мы закладываем inductive bias на уровне обработки данных. Таким образом мы сглаживаем проблемы, присущие наивным подходам. Данный способ токенизации широко используется в современных языковых моделях.



[Источник](#)

Это интересно

Построение словаря – важная и сложная задача. Например, для больших моделей есть эмпирические правила которые связывают число параметров, размер выборки и размер словаря, вы познакомитесь с ними на следующих занятиях.

Кроме того, иногда задача построения достаточно «хорошего» словаря может требовать особых знаний. Предположим, что ваша задача связана с органической химией. Для начала нужно понять, как превратить наши данные в числа. Например, молекулы и белки можно обрабатывать как графы, что требует сложных архитектур.

С другой стороны, для каждого химического соединения существует представление в виде строки, которая называется [SMILES](#). Например, метил (CH_3) записывается как **CH3**, а этилен (C_2H_4) – **C=C** (так как присутствует двойная связь, а водород не пишется явно, если его можно вывести из валентности).

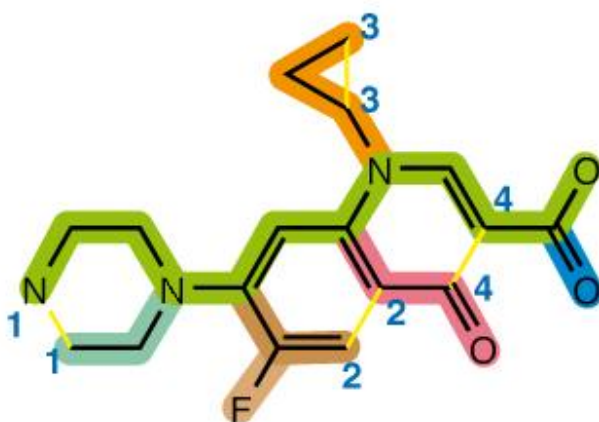
Соответственно, если перед вами стоит задача генерации химических соединений, то намного проще будет генерировать строки, а не графы – остается лишь выбрать подходящий словарь.

Самый простой вариант – работать на уровне символов. Тогда вы сможете получить совершенно любую строку, однако в процессе генерации могут возникнуть некорректные строки (не каждая строка соответствует химическому соединению). Вы будете тратить мощность модели и время обучения на изучение структуры SMILES и корректных строк.

Продвинутый вариант – узнать, какую именно вы задачу решаете на практике. Например, если это синтез новых соединений, то часто вы знаете из каких «кусков»

состоит результат. Например, соседний отдел коллег-химиков может подсказать вам, какие связи встречаются, встречаются ли у вас бензольные кольца и так далее. Благодаря этим знаниям вы можете составить словарь, из которого вы уже не сможете получить любое соединение, но любое сгенерированное соединение будет корректным.

Такая работа со словарем может превратить нерабочий метод в рабочий, так как закладывает знания в обработку данных и значительно упрощает решение задачи.



N1CCN(CC1)C(C(F)=C2)=CC(=C2C4=O)N(C3CC3)C=C4C(=O)O

[Источник](#)

Общая задача построения эмбедингов

Примечание: В русскоязычных текстах принято употреблять термин "векторное представление". Будьте внимательные в ваших курсовых, а также в любых других работах!

Дано: $D = \{w_1, w_2, \dots, w_{L_d}\}_{d=1}^{d=N}$ – корпус текста, $w_i \in W$ – словарь всевозможных слов (единиц конструкции предложения).

Найти: векторное представление $v_w \in R^m$ для каждого слова w , где $m \ll |W|$

Вопрос: Что мы хотим получить от векторных представлений?

Ответ:

Далее мы будем считать, что хорошие представления будут иметь:

1. Соответствие близости по смыслу слов к близости по расстоянию между векторами.
2. Интерпретируемые арифметические операции над словами в пространстве эмбедингов.
3. Хорошее качество при решении конечной задачи.

На самом деле, мы хотим не просто уменьшить вектора, но и иметь векторное пространство с операциями сохраняющим "смысл" слов.

Если мы сможем построить такие векторные представления, то **сразу решим задачи**:

1. Поиск близких слов, синонимов и т.п.
2. Получение представление документа, которое будет использоваться в других задачах машинного обучения.
3. Использование в качестве фиксированного представления в сложной архитектуре (например, рекуррентной сети*)
4. Использование для инициализации представлений в сложной архитектуре (снова рекуррентной сети*)

Все эти аспекты рассмотрим на практике

*Про рекуретные сети вы узнаете на следующей неделе

Гипотеза дистрибутивности

Сделаем некоторые предположения, чтобы постараться хоть как-то подойти к нашей задаче построения хороших эмбедингов. Благо, это уже было сделано за нас более полувека тому назад.

1. **Основная гипотеза, формулировка 1** ([Zellig S. Harris, 1954](#), примерно 8400 цитирований):

Слова, встречающиеся с одними и теми же словами, имеют схожее значение.

2. **Основная гипотеза, формулировка 2** ([J. Firth, 1957](#), примерно 5700 цитирований):

Слово характеризуется словами, с которыми оно встречается.

Вопрос: Как вы считаете, близка ли к реальности эта гипотеза? Какие примеры вы знаете, которые противоречат гипотезе?

Ответ:

В целом, гипотеза близка к реальности, однако есть исключения:

- В одном контексте часто могут встречаться слова с диаметрально противоположным значением
- В одном контексте могут встречаться различные имена собственные
- Слова, имеющие несколько различных значений, довольно тяжело характеризовать соседними словами
- В некоторых языках отрицательная частица ставится в конец предложения, хоть она и связана с глаголом в начале (например, частица *nicht* в немецком)

Основные подходы к построению эмбедингов

Подход #0: BOW и Tf-Idf

В базовом курсе машинного обучения вы познакомились с базовыми методами кодирования слов – **BoW** и **Tf-Idf**. Они опираются на частоту встречаемости слов в корпусе (**count-based** методы).

$$o_{cat} = \begin{pmatrix} 0 \\ 0 \\ \dots \\ 0 \\ 1 \\ 0 \\ \dots \\ 0 \\ 0 \end{pmatrix} \begin{matrix} \text{"a"} \\ \text{"abbreviations"} \\ \\ \text{"cast"} \\ \text{"cat"} \\ \text{"cate"} \\ \\ \text{"zoology"} \\ \text{"zoom"} \end{matrix} \quad o_{dog} = \begin{pmatrix} 0 \\ 0 \\ \dots \\ 0 \\ 1 \\ 0 \\ \dots \\ 0 \\ 0 \end{pmatrix} \begin{matrix} \text{"a"} \\ \text{"abbreviations"} \\ \\ \text{"do"} \\ \text{"dog"} \\ \text{"donut"} \\ \\ \text{"zoology"} \\ \text{"zoom"} \end{matrix}$$

Вопрос: Какие проблемы имеются у данных подходов?

Ответ:

Основные проблемы:

- Получившиеся векторные представления имеют размерность, равную размеру словаря, что очень много
- В векторы никак не зашита информация о порядке слов в тексте, об их семантике и контексте

Подход #1: Более умные Count-based методы

Наша цель сохранить в векторах смыслы слов и контекста, а также уменьшить размерность. Если слова зависят друг от соседних слов (в каком-то контексте), тогда попробуем зашить эту информацию в итоговые векторы.

Для начала формально определим, что значит "слово c является контекстом слова w ".

Под контекстом слова w будем понимать k слов слева и k слов справа от w , обозначим его $C(w)$. Таким образом, фраза выше означает то, что $c \in C(w)$.

Обозначим за $n_{w,c}$ количество раз, когда слово c являлось контекстом слова w и определим следующие величины ([Peter D. Turney and Patrick Pantel, 2010](#)):

$$p(w, c) = \frac{n_{w,c}}{\sum_{i=1}^N \sum_{j=1}^N n_{i,j}}$$
$$p(w*) = \frac{\sum_{c=1}^N n_{w,c}}{\sum_{i=1}^N \sum_{j=1}^N n_{i,j}}$$
$$p(*c) = \frac{\sum_{w=1}^N n_{w,c}}{\sum_{i=1}^N \sum_{j=1}^N n_{i,j}}$$

Здесь:

- $p(w, c)$ – вероятность встретить слово c в контексте слова w
- $p(w*)$ – вероятность встретить слово w в качестве слова, контекст которого мы рассматриваем
- $p(*c)$ – вероятность встретить слово c в произвольном контексте

Запишем, чему равна взаимная информация двух слов (pointwise mutual information/PMI), что по смыслу отвечает на вопрос: "Насколько одно слово влияет на другое?".

$$PMI(w, c) = \log \frac{p(w, c)}{p(w*)p(*c)}$$

Если раскрыть счетчики в формулах вероятностей, то получим:

$$PMI(w, c) = \log \frac{n_{w,c} \cdot N}{n_{w*} n_{*c}}$$

Здесь N – количество рассмотренных пар (w, c) .

Рассмотрим позитивную взаимную информацию (Positive PMI/PPMI):

$$PPMI(w, c) = \max(0, PMI(w, c))$$

Составим матрицу $X \in \mathbb{R}^{|W| \times |W|}$, где $X_{w,c} = PPMI(w, c)$.

Вопрос: Какая матрица это будет? Как это соотносится с реальным языком?

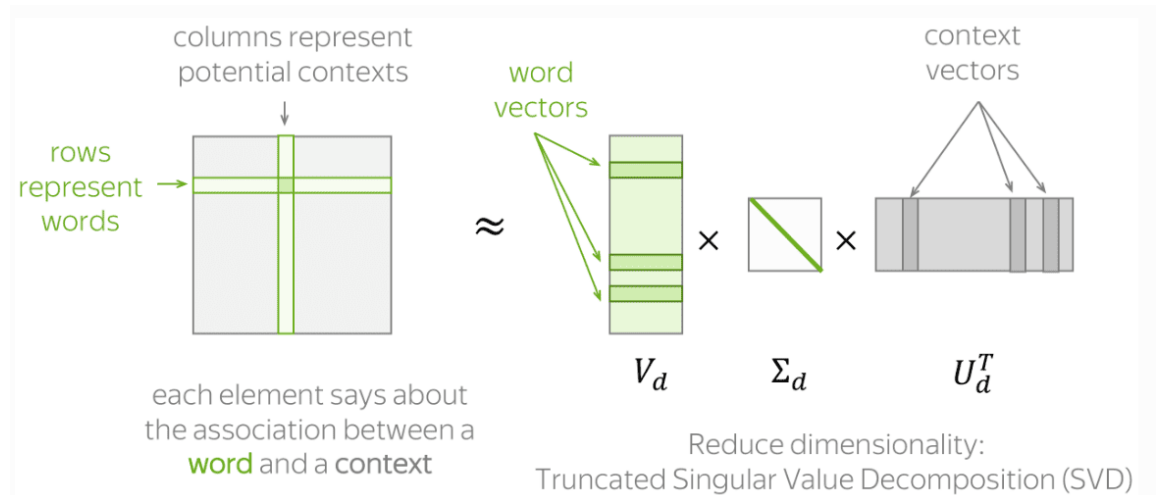
Ответ:

Получим сильно разреженную матрицу, содержащую информацию о взаимной встречаемости слов, так как $X_{i,j} > 0$, если слова i, j встречаются в одном контексте чаще, чем по отдельности.

*Взаимная информация прошла из теории информации, любопытные могут прочитать об этом подробнее в соответствующих книгах :)

Итог

Мы получили очень большую разреженную матрицу, в которой содержится полезная информация, теперь давайте уменьшим ее размер. Для этого воспользуемся методами понижения размерности, например, SVD разложением.



Источник

Когда мы получили матрицу взаимодействий мы уже решили прикладную задачу. Задача уменьшения размерности матрицы с сохранением "информации" является задачей методов оптимизации (alternating least squares, ...) или линейной алгебры (SVD, TT-SVD, ...). Посмотрим на подход, в котором мы строим наши матрицы путем оптимизации некоторого функционала.

Glove: Global Vectors for Word Representation

Jeffrey Pennington et al., 2014, примерно 50 тысяч цитирований

Зададим матрицы U и V , которые будут содержать вектора слов и контекста соответственно.

Обозначим $P_{w,c} := P(c \mid w)$ и будем учить U, V так, чтобы

$$u_w^T v_c \approx \log P_{w,c} = \log \frac{n_{w,c}}{n_{w*}}$$

Это будет означать, что векторы, соответствующие семантически схожим словам, будут близки с точки зрения скалярного произведения. Для этого будем решать следующую оптимизационную задачу:

$$\mathcal{L} = \sum_{w \in W} \sum_{c \in W} F(n_{w,c}) \left(u_w^T v_c + b_w + \hat{b}_c - \log n_{w,c} \right)^2 \rightarrow \min_{U, V, b_w, \hat{b}_c} \quad (1)$$

$$F(n) = \begin{cases} \frac{n_{w,c}^\alpha}{n_{max}^\alpha}, & \text{if } n_{w,c} < n_{max} \\ 1, & \text{иначе} \end{cases} \quad (2)$$

$$\alpha = \frac{3}{4}, n_{max} = 100, \quad \text{гиперпараметры из статьи.} \quad (3)$$

- Чем чаще вместе встречаются слова w и c , тем больший вклад в функционал вносит соответствующее им слагаемое $\Rightarrow$ тем важнее достичь близости $u_w^T v_c + b_w + \hat{b}_c$ и $\log n_{w,c}$.
- Слагаемые $b_w, \hat{b}_c$ возникают в силу того, что $\log P_{w,c} = \log n_{w,c} - \underbrace{\log n_{w*}}_{\text{не зависит от } c}$ (b_w

для симметрии). Да, мы могли бы собрать статистику n_{w*} по корпусу так же, как мы собираем $n_{w,c}$, и не учить смещения, но тогда пропала бы симметрия при замене слова и контекста, а авторы статьи хотят ее сохранить для того, чтобы в качестве финального эмбединга брать сумму строк обеих матриц. Можно ввести следующую интуицию: u_w, v_c отвечают за смысловую связь, $b_w, \hat{b}_c$ — за частоту встречаемости слов в корпусе.

Алгоритм обучения:

- Проходимся по корпусу и строим матрицу $X_{ij} = n_{i,j}$
- Инициализируем U, V
- Минимизируем функцию потерь градиентными методами
- В качестве финального эмбединга можно брать $u_w + v_w$

Подробнее можно почитать в статье, интуиция и вывод функционала очень интересен и любопытен, **хороший пример** как люди старались заложить ожидания от решения в задачу оптимизации.

На практике Glove используется, но **работает хуже** более продвинутых методов, которые мы рассмотрим ниже.

Общие замечания к методам, основанным на подсчете

Преимущества:

- Неплохое качество в некоторых задачах (но нужно уметь настраивать)

- Маленькая размерность
- Близким словам соответствуют близкие вектора

Недостатки

- Нет хорошего механизма обработки новых слов на тесте
- **Основной минус:** необходимо собирать огромную (но разреженную!) матрицу совстречаемостей для обучения

Так как нет возможности добавлять новые слова и необходимость хранения матрицы, этот подход **не** подходит для нейронных сетей и больших датасетов. Нам нужны **масштабируемые** методы.

Подход #2: Prediction-based (word2vec)

[Tomas Mikolov et al., 2013](#), примерно 52 тысячи цитирований

Строить эмбединги хоть как-нибудь хорошо мы научились выше. Теперь наша цель обновлять их "на ходу", чтобы адаптировать под новые данные и учить **масштабируемые модели**.

Идея:

Будем обучать модель «воспроизводить» локально гипотезу Фёрса (версия 2). Есть **два** основных подхода:

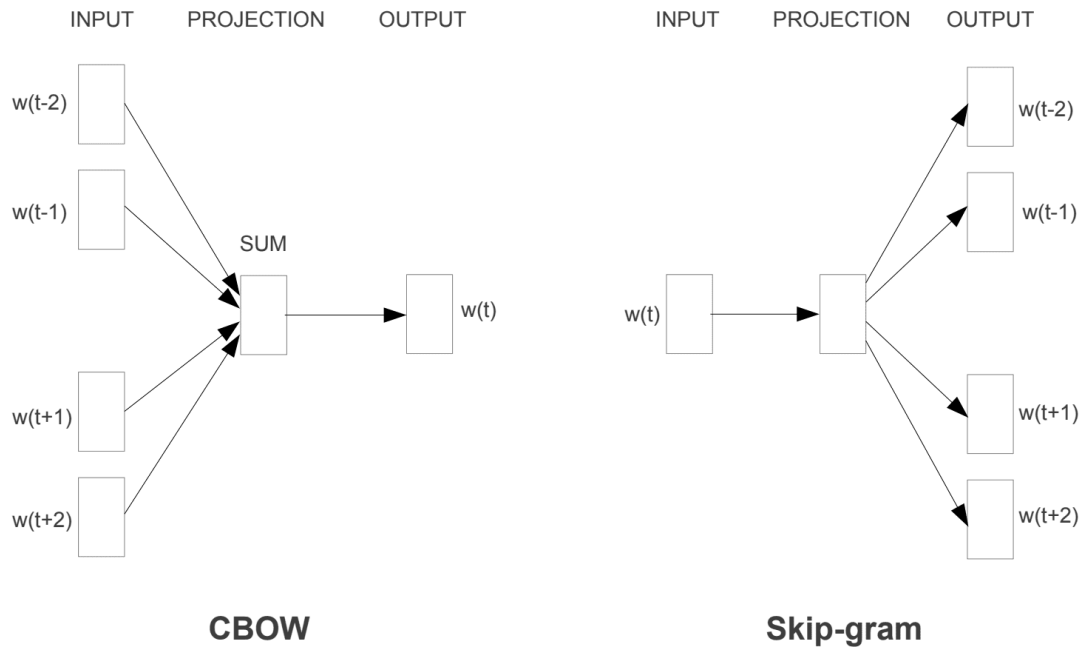
- Модель CBOW – по словам контекста необходимо предсказать центральное слово
- Модель Skip-gram – по центральному слову, необходимо предсказать каждое из слов контекста

Модель CBOW (continuous bag of words)

Если слово характеризуется соседними словами, значит мы можем **предсказать слово по контексту**.

Модель Skip-gram

Если слово характеризуется соседними словами, значит мы можем **предсказать контекст по слову**.



Источник

Далее будем считать, что есть словарь V , матрица входных эмбеддингов $W \in \mathbb{R}^{|V| \times d}$, матрица выходных эмбеддингов $\hat{W} \in \mathbb{R}^{|V| \times d}$. Каждое слово имеет входной вектор w_i и выходной вектор $\hat{w}_i$.

Почему две матрицы:

- Не накладываем дополнительных ограничений (симметричность) на входные данные
- Проще считать градиенты – быстрее обучаемся

Функционал CBOW

Будем считать, что $C(v_i)$ контекст слова v_i , например k слов слева и справа.

$$\sum_i^N \log p(v_i | C(v_i)) \rightarrow \max_{W, \hat{W}}$$

$$p(v_i | C(v_i)) = \text{softmax}(h \hat{W}^T)_{v_i}$$

$$h = \frac{1}{|C(v_i)|} \sum_{v \in C(v_i)} W_v$$

Функционал Skip-Gram

$$\sum_i^N \sum_{v \in C(v_i)} \log p(v | v_i) \rightarrow \max_{W, \hat{W}'}$$

$$p(v | v_i) = \text{softmax}(W_v \hat{W}'^T)_{v_i}$$

В качестве эмбедингов выбираются w_i или $w_i + \hat{w}_i$.

Замечание

- На практике skip-gram намного лучше моделируют редкие слова
- Skip-gram на практике намного медленнее CBOW
- Есть методы ускорения обучения: Negative sampling, Hierarchical softmax из работы [Tomas Mikolov et al., 2013](#), примерно 50 тысяч цитирований

Model	Dimension	Size	Semantics	Syntax	Total
ivLBL	100	1.5B	55.9	50.1	53.2
HPCA	100	1.6B	4.2	16.4	10.8
GloVE	100	1.6B	67.5	54.3	60.3
SG	300	1B	61	61	61
CBOW	300	1.6B	16.1	52.6	36.1
vLBL	300	1.5B	54.2	64.8	60.0
ivLBL	300	1.5B	65.2	63.0	64.0
GloVe	300	1.6B	80.8	61.5	70.3
SVD	300	6B	6.3	8.1	7.3
SVD-S	300	6B	36.7	46.6	42.1
SVD-L	300	6B	56.6	63.0	60.1
CBOW	300	6B	63.6	67.4	65.7
SG	300	6B	73.0	66.0	69.1
GloVe	300	6B	77.4	67.0	71.7
CBOW	1000	6B	57.3	68.9	63.7
SG	1000	6B	66.1	65.1	65.6
SVD-L	300	42B	38.4	58.2	49.2
GloVe	300	42B	81.9	69.3	75.0

[Источник таблички](#)

Общие замечания к методам основанным на предсказании

Преимущества:

- Маленькая размерность
- Близким словам соответствуют близкие вектора
- Обучаются градиентно по датасету

Недостатки

- Плохой механизм обработки новых слов на тесте
- Требуют большего корпуса чем count-based модели

Так как нет возможности добавлять новые слова и необходимость хранения матрицы, этот подход **не** подходит для нейронных сетей и больших датасетов. Нам нужны **масштабируемые** методы.

Удачное расширение skip-gram – модель **FastText** ([Piotr Bojanowski et al., 2017](#), примерно 15 тысяч цитирований). Для этого предположим, что эмбединг слова представляются суммой эмбедингов его буквенных n-грамм, и будем учить именно их. Это удачно работает с **лингвистической точки зрения**: смысл слова определяется приставкой, корнем, суффиксом и окончанием. Кроме того, мы теперь можем обрабатывать Out-Of-Vocabulary слова, так как каждое слово состоит из набора слогов.

Способы работы с эмбедингами

Итак, мы научились строить эмбединги, которые соответствуют нашим предположениям об их свойствах. Давайте теперь посмотрим, как мы можем работать с полученными векторными представлениями.

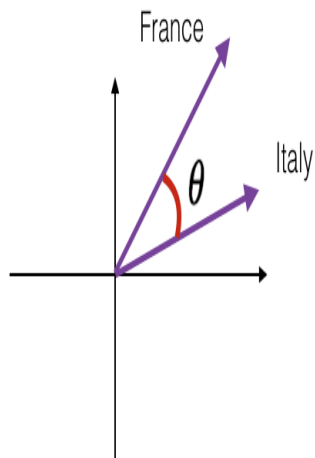
Измерение близости слов

Для измерения сходства двух слов можно использовать предположение о близости по некоторой метрике их эмбедингов.

Рассмотрим косинусную меру сходства:

$$\text{CosineSimilarity}(u, v) = \frac{u \cdot v}{||u||_2 ||v||_2} = \cos(\theta),$$

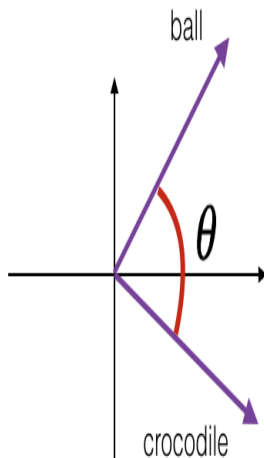
θ – угол между векторами u и v .



France and Italy are quite similar

θ is close to 0°

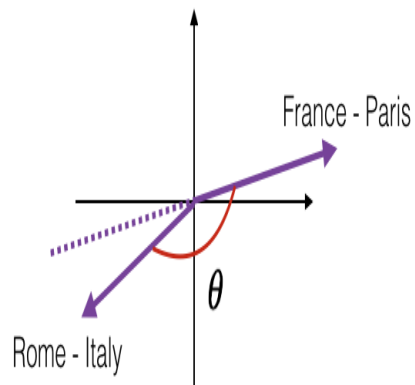
$$\cos(\theta) \approx 1$$



ball and crocodile are not similar

θ is close to 90°

$$\cos(\theta) \approx 0$$



the two vectors are similar but opposite
the first one encodes (city - country)
while the second one encodes (country - city)

θ is close to 180°

$$\cos(\theta) \approx -1$$

[Источник](#)

Арифметика в поле эмбедингов

Основное свойство, которое мы хотим получить от эмбедингов -- это соответствие математических операций над векторами слов аналогичным им смысловым операциям над самими словами. Давайте рассмотрим изображения

Word2Vec

На векторах word2vec можно проводить векторную арифметику:

$$\mathbf{v}(\text{king}) - \mathbf{v}(\text{man}) + \mathbf{v}(\text{woman}) \approx \mathbf{v}(\text{queen})$$

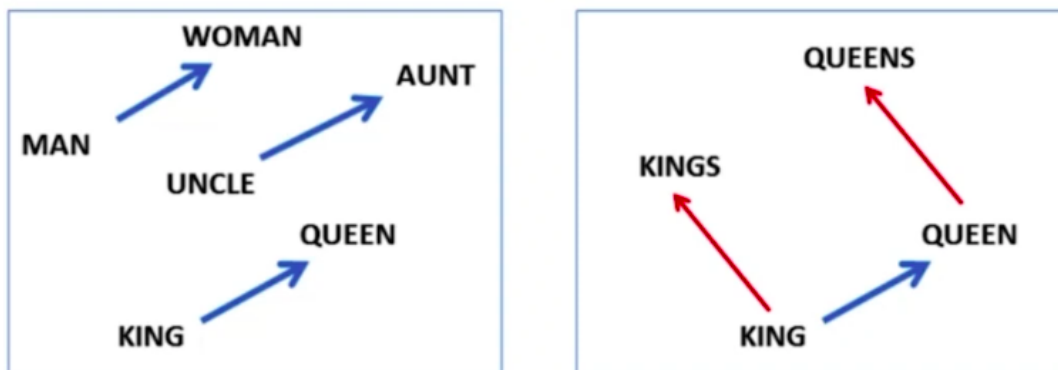
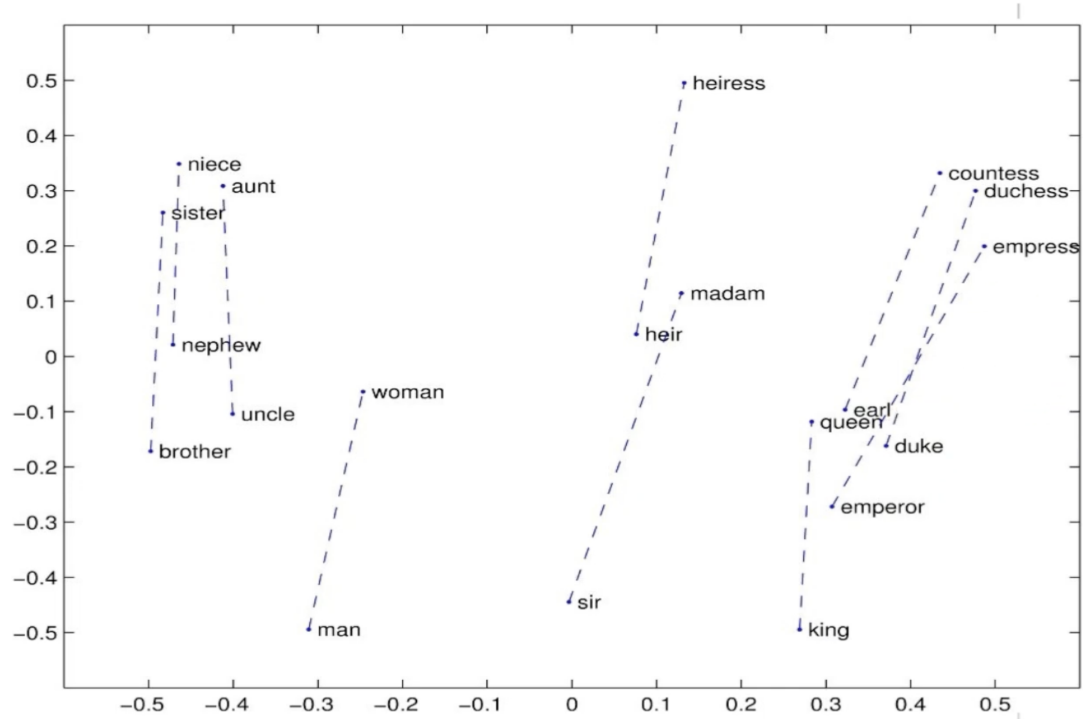


Иллюстрация эмбедингов после
понижения размерности

[Источник](#)

Word2Vec



[Источник](#)

Сравнение методов

Мы сегодня рассмотрели достаточно простые методы создания эмбедингов, но достаточно эффективные, если посмотреть на изображение ниже, то можно увидеть незначительное отставание GloVe даже от XXL моделей. В общем данные подходы являются очень сильными бейзлайнами.

Num. Datasets (→)	Class. 12	Clust. 11	PairClass. 3	Rerank. 4	Retr. 15	STS 10	Summ. 1	Avg. 56
<i>Self-supervised methods</i>								
Glove	57.29	27.73	70.92	43.29	21.62	61.85	28.87	41.97
Komninos	57.65	26.57	72.94	44.75	21.22	62.47	30.49	42.06
BERT	61.66	30.12	56.33	43.44	10.59	54.36	29.82	38.33
SimCSE-BERT-unsup	62.50	29.04	70.33	46.47	20.29	74.33	31.15	45.45
<i>Supervised methods</i>								
SimCSE-BERT-sup	67.32	33.43	73.68	47.54	21.82	79.12	23.31	48.72
coCondenser-msmarco	64.71	37.64	81.74	51.84	32.96	76.47	29.50	52.35
Contriever	66.68	41.10	82.53	53.14	41.88	76.51	30.36	56.00
SPECTER	52.37	34.06	61.37	48.10	15.88	61.02	27.66	40.28
LaBSE	62.71	29.55	78.87	48.42	18.99	70.80	31.05	45.21
LASER2	53.65	15.28	68.86	41.44	7.93	55.32	26.80	33.63
MiniLM-L6	63.06	42.35	82.37	58.04	41.95	78.90	30.81	56.26
MiniLM-L12	63.21	41.81	82.41	58.44	42.69	79.80	27.90	56.53
MiniLM-L12-multilingual	64.30	37.14	78.45	53.62	32.45	78.92	30.67	52.44
MPNet	65.07	43.69	83.04	59.36	43.81	80.28	27.49	57.78
MPNet-multilingual	67.91	38.40	80.81	53.80	35.34	80.73	31.57	54.71
OpenAI Ada Similarity	70.44	37.52	76.86	49.02	18.36	78.60	26.94	49.52
SGPT-125M-nli	61.46	30.95	71.78	47.56	20.90	74.71	30.26	45.97
SGPT-5.8B-nli	70.14	36.98	77.03	52.33	32.34	80.53	30.38	53.74
SGPT-125M-msmarco	60.72	35.79	75.23	50.58	37.04	73.41	28.90	51.23
SGPT-1.3B-msmarco	66.52	39.92	79.58	54.00	44.49	75.74	25.44	56.11
SGPT-2.7B-msmarco	67.13	39.83	80.65	54.67	46.54	76.83	27.87	57.12
SGPT-5.8B-msmarco	68.13	40.35	82.00	56.56	50.25	78.10	24.75	58.81
SGPT-BLOOM-7.1B-msmarco	66.19	38.93	81.90	55.65	48.21	77.74	24.99	57.44
GTR-Base	65.25	38.63	83.85	54.23	44.67	77.07	29.67	56.19
GTR-Large	67.14	41.60	85.33	55.36	47.42	78.19	29.50	58.28
GTR-XL	67.11	41.51	86.13	55.96	47.96	77.80	30.21	58.42
GTR-XXL	67.41	42.42	86.12	56.65	48.48	78.38	30.64	58.97
ST5-Base	69.81	40.21	85.17	53.09	33.63	81.14	31.39	55.27
ST5-Large	72.31	41.65	84.97	54.00	36.71	81.83	29.64	57.06
ST5-XL	72.84	42.34	86.06	54.71	38.47	81.66	29.91	57.87
ST5-XXL	73.42	43.71	85.06	56.43	42.24	82.63	30.08	59.51

Table 1: Average of the main metric (see Section 3.2) per task per model on MTEB English subsets.

Источник

Table 6: Extrinsic Evaluation

Embedding Type	NER			SA			TC		
	P	R	F1	P	R	F1	P	R	F1
GloVe (768d)	0.833	0.809	0.820	0.381	0.358	0.346	0.908	0.908	0.908
Word2Vec SG (768d)	0.882	0.837	0.858	0.447	0.436	0.415	0.920	0.92	0.919
FastText (768d)	0.881	0.852	0.866	0.440	0.434	0.391	0.918	0.918	0.917
BERT	0.921	0.928	0.925	0.508	0.500	0.471	0.937	0.938	0.937

Источник

Применение в реальных задачах

В современном мире все трубят о больших языковых моделях, поэтому будем рассматривать именно задачу генерации текста. Формально мы можем записать это следующим образом:

Дано: Выборка документов $\mathcal{D} = \{d_1, \dots, d_N\}$. Каждый документ содержит слова $w_1, \dots, w_{L_d}$, где L_d число слов в документе d .

Найти: Для любого множества слов $(w_1, \dots, w_k)$ оценить вероятность следующего слова в предложении, то есть найти условное распределение:

$$p(w|w_1, \dots, w_k)$$

Нетрудно понять, что при инференсе нашей модели наличие в эмбедингах информации о семантике слов критически важно. Без него модель может не учесть важные связи между словами в последовательности и, как следствие, выдать неправильное распределение, сломав генерацию.

Эмбединги в PyTorch

Самый простой и лучший (в смысле некоторой обучаемой модели, а не семантической интерпретируемости) способ обучить эмбединги – инициализировать векторы случайным образом, а затем пропускать по ним градиент, т.е. рассматривать как обучаемый параметр при прямом проходе сети. Таким образом мы можем выучить наши эмбединги под конкретную задачу. Также отлично работает инициализация известными эмбедингами, теми же Glove и FastText.

```
torch.nn.Embedding(  
    num_embeddings: int,  
    embedding_dim: int,  
    padding_idx: Optional[int] = None,  
    max_norm: Optional[float] = None,  
    norm_type: float = 2.0,  
    scale_grad_by_freq: bool = False,  
    sparse: bool = False,  
    _weight: Optional[torch.Tensor] = None,  
    _freeze: bool = False,  
    device=None,  
    dtype=None,  
) -> None
```

Инициализация Эмбедингов: класс `torch.nn.Embedding`

- `num_embeddings` (int) – размер словаря (количество слов, требующих эмбедингов). Включает в себя спец. слова. Можно передавать `vocab.__len__`
- `embedding_dim` (int) – размерность изучаемого представления.

- `padding_idx` (int, optional) – если указан, то вектор соответствующий этому токenu обучаться не будет, по умолчанию будет равен 0.

Важно! `padding_idx` используется для обособления начала/конца строки. Векторы других спец символов, например `<unk>`, обучаться будут.

- `max_norm` (float, optional) – максимальная норма векторного представления, если таковая указана.
- `norm_type` (float, optional) – параметр нормы (p-norm).
- `scale_grad_by_freq` (bool, optional) – Если True, то градиенты будут считаться с учётом частоты слова в мини-батче.
- `sparse` (bool, optional) – Если True, то градиент по матрице эмбедингов будет вычисляться в sparse виде. (Нужно, например, если вы захотели дообучить некоторые из приведённых ранее эмбедингов)